BÀI TẬP THỰC HÀNH: XÂY DỰNG AI NHẬN DIỆN VẬT THỂ VỚI CIFAR-10

I. ĐỀ BÀI (TÌNH HUỐNG THỰC TẾ)

Bối cảnh: Bạn đang làm việc tại một công ty phát triển Camera thông minh giao thông và an ninh. Đội ngũ kỹ sư cần một mô hình AI cốt lõi có khả năng nhìn vào một bức ảnh độ phân giải thấp và phân loại nhanh chóng vật thể trong ảnh đó là phương tiện giao thông hay động vật để đưa ra cảnh báo.

Nhiệm vụ của bạn: Xây dựng một mạng Nơ-ron Tích chập (CNN) để phân loại bộ dữ liệu CIFAR-10.

Thông tin Bộ dữ liệu (CIFAR-10):

- Gồm 60.000 ảnh màu RGB, kích thước 32×32 pixel.
- Dữ liệu được chia thành **10 lớp** rõ rệt: Máy bay (Plane), Ô tô (Car), Chim (Bird), Mèo (Cat), Hươu (Deer), Chó (Dog), Ếch (Frog), Ngựa (Horse), Tàu thủy (Ship), Xe tải (Truck).

Mục tiêu cần đạt: Độ chính xác trên tập kiểm tra (Test Accuracy) tối thiểu **70%**. *(Lưu ý: Với ảnh màu phức tạp, đạt được 70% bằng một mạng CNN tự xây từ đầu đã là một kết quả rất tốt!)*

II. HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC

Bước 1: Khởi tạo và tải dữ liệu

Gọi dữ liệu CIFAR-10 từ thư viện Keras.

Python
```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# Tải bộ dữ liệu CIFAR-10
cifar10 = tf.keras.datasets.cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Danh sách 10 lớp (class names)
class_names = ['Máy bay', 'Ô tô', 'Chim', 'Mèo', 'Hươu',
               'Chó', 'Ếch', 'Ngựa', 'Tàu thủy', 'Xe tải']
```

Bước 2: Tiền xử lý dữ liệu (Preprocessing)

Vì CIFAR-10 tải về đã có sẵn định dạng khối 3D là $(32, 32, 3)$ (cao, rộng, 3 kênh màu RGB), nên chúng ta không cần lệnh reshape. Bạn chỉ cần thực hiện việc Chuẩn hóa (Normalization).

Python
```
# Chuẩn hóa giá trị pixel từ [0, 255] về [0, 1]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print("Kích thước dữ liệu huấn luyện:", x_train.shape) # Sẽ in ra (50000, 32, 32, 3)
```
Bước 3: Thiết kế Kiến trúc CNN (Model Design)

Ảnh màu RGB chứa nhiều thông tin phức tạp (kết cấu lông, viền kim loại, màu trời...), do đó bạn cần một mạng sâu hơn (3 Block Tích chập) và thêm kỹ thuật Dropout để tránh học vẹt.

Python
```
model = models.Sequential()

# Block 1: Trích xuất đặc trưng mức thấp (cạnh, màu sắc)
model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))

# Block 2: Trích xuất đặc trưng mức trung (hình khối)
model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))

# Block 3: Trích xuất đặc trưng mức cao (bộ phận phức tạp như bánh xe, mắt chó)
model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))

# Khối phân loại
model.add(layers.Flatten())
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dropout(0.5)) # Tắt ngẫu nhiên 50% nơ-ron để chống Overfitting
model.add(layers.Dense(10, activation='softmax'))

model.summary()
```
Bước 4: Biên dịch và Huấn luyện (Compile & Train)

Bộ dữ liệu này phức tạp hơn nên mô hình sẽ cần nhiều thời gian học hơn. Hãy thử đặt số vòng lặp (epochs) là 15.

Python
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Quá trình này có thể mất vài phút tùy tốc độ máy tính/Colab
history = model.fit(x_train, y_train, epochs=15,
                    validation_split=0.2, batch_size=64)
```
Bước 5: Đánh giá và Vẽ biểu đồ

Với bài toán khó như CIFAR-10, việc vẽ biểu đồ Loss và Accuracy là bắt buộc để xem mô hình có bị hiện tượng "học vẹt" hay không.

Python
```
# Đánh giá trên tập kiểm tra
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f'\nĐộ chính xác trên tập kiểm tra: {test_acc*100:.2f}%')

# Vẽ biểu đồ Accuracy
plt.plot(history.history['accuracy'], label='Accuracy (Train)')
plt.plot(history.history['val_accuracy'], label='Accuracy (Validation)')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.title('Biểu đồ Độ chính xác qua các vòng lặp')
plt.show()
```

III. THỬ THÁCH NÂNG CAO (LẤY ĐIỂM CỘNG)

Nếu mô hình của bạn bị mắc kẹt ở độ chính xác 65-70% và biểu đồ Validation Accuracy đi ngang, hãy áp dụng "vũ khí bí mật" đã học trong bài: Tăng cường dữ liệu (Data Augmentation) .

Yêu cầu: Hãy tự tìm hiểu cách sử dụng `tf.keras.preprocessing.image.ImageDataGenerator` để tạo ra các bức ảnh bị lật ngang (Horizontal Flip) hoặc dịch chuyển nhẹ. Tích hợp nó vào quá trình huấn luyện và xem độ chính xác có vượt qua ngưỡng 75% không nhé!